

CDS View 課程練習 7：聚合與分組

Lecture

這一課要教什麼

到目前為止建的 CDS View，一列輸入永遠對應一列輸出（就算加了計算欄位、Association，也只是「豐富化」每一列，列數不會變）。這一課要教另一種完全不同的查詢型態：**聚合 (Aggregation)**——把多列資料彙總成更少的列，例如「每家航空公司總共有幾筆訂位、平均票價多少」，這種問題直接查明細表回答不了，要用 **GROUP BY** 搭配聚合函數。

語法元素講解

① **聚合函數**：CDS 支援標準 SQL 常見的聚合函數：

函數	用途
COUNT(*)	計算群組內的列數
SUM(<欄位>)	加總
AVG(<欄位>)	平均
MIN(<欄位>) / MAX(<欄位>)	最小值 / 最大值（這一課沒用到，但語法相同）

② **GROUP BY**：決定「依什麼欄位分組」，沒有出現在 **GROUP BY** 子句、也沒有被聚合函數包起來的欄位，不能直接放進欄位清單（這點跟一般 SQL 規則一致）：

```
define view ZC_CDS07_ROUTE_STATS
as select from ZI_CDS07_FLIGHT
{
  key carrid,
  count(*)      as FlightCount,
  sum(seatsocc) as TotalSeatsOccupied,
  avg(price)    as AvgPrice
}
group by carrid
```

⚠ **group by** 子句的位置：跟 cds04 學到的 **WHERE** 子句一樣，寫在整個欄位清單 **{ }** 的後面。

③ **@DefaultAggregation**：跟前面兩點不同，這個 annotation 不是「執行聚合」，而是替一個還沒被聚合的原始欄位標記「如果之後有人要聚合這個欄位，預設該用哪種聚合方式」——這是給分析工具（例如 Fiori Elements 的 Analytical List Page、或第三方 BI 工具）看的提示，工具讀到這個 annotation，就知道使用者在畫面上拖曳這個欄位時該自動套用哪種彙總邏輯，不用每次都手動選：

```
@DefaultAggregation: #SUM
seatsocc,
```

```
@DefaultAggregation: #AVG
price,
```

這一課的 **ZI_CDS07_FLIGHT** (Layer 1 , 還是明細層級、一列一筆訂位) 示範了這個 annotation 的語法 ; **ZC_CDS07_ROUTE_STATS** (Layer 2 , 真正執行聚合) 則示範了「動手把資料彙總起來」這件事本身怎麼寫。兩者**是互補、不是二選一** : **@DefaultAggregation** 讓「消費明細層級 View 的下游工具」知道怎麼聚合 ; **GROUP BY** 是你自己在 CDS View 裡明確定義好的聚合結果。

完整範例

Layer 1 : ZI_CDS07_FLIGHT (明細層級 , 帶聚合提示)

```
@AbapCatalog.sqlViewName: 'ZICDS07FLGT'
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'CDS07: Flight Interface View with Default Aggregation Hints'
define view ZI_CDS07_FLIGHT
  as select from sflight
{
  key carrid,
  key connid,
  key fldate,

  @DefaultAggregation: #SUM
  seatsocc,

  @DefaultAggregation: #AVG
  price,

  currency
}
```

Layer 2 : ZC_CDS07_ROUTE_STATS (聚合層級 , 依 carrid 分組)

```
@AbapCatalog.sqlViewName: 'ZCCDS07RTST'
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'CDS07: Route Statistics (Aggregated)'
define view ZC_CDS07_ROUTE_STATS
  as select from ZI_CDS07_FLIGHT
{
  key carrid,

  count(*)           as FlightCount,
  sum(seatsocc)       as TotalSeatsOccupied,
  avg(price)         as AvgPrice
}
```

```
}  
group by carrid
```

查詢 **ZC_CDS07_ROUTE_STATS** 會拿到「每家航空公司一列」的結果（這系統測試資料是 8 家航空公司，就是 8 列），而不是像明細表一樣有幾百列訂位紀錄。

跟應用層聚合的效能對比

在有 CDS 聚合能力之前，很多報表程式的寫法是：先用 Open SQL 把明細資料整批撈到 Application Server，再用 **LOOP** 迴圈自己累加：

```
SELECT carrid, seatsocc, price FROM sflight WHERE carrid = 'AA' INTO TABLE  
@DATA(lt_raw).  
LOOP AT lt_raw INTO DATA(ls_raw).  
    lv_count = lv_count + 1.  
    lv_sum_seats = lv_sum_seats + ls_raw-seatsocc.  
    lv_sum_price = lv_sum_price + ls_raw-price.  
ENDLOOP.  
lv_avg_price = lv_sum_price / lv_count.
```

這種寫法**結果是對的**（這一課的驗證程式證實兩種算法算出來的數字完全一致），但有一個實際的成本差異：**應用層聚合要把每一筆明細資料都從資料庫傳到 Application Server**，聚合層級的 **CDS View** 只需要傳回「已經算好的彙總結果」。這一課的驗證程式量測了實際筆數：查 **ZC_CDS07_ROUTE_STATS**（已聚合）只需要傳回 8 列（每家航空公司一列）；如果要在應用層自己算，光是 **AA** 一家航空公司的明細就要傳 25 列，資料量越大、航空公司越多，這個差距會被放大——**這不是憑空猜測的效能理論，是這一課實測量到的具體筆數差異**（詳見下方驗證方式）。

⚠ 誠實的澄清：這一課用的測試資料量很小（總共只有幾百筆），量測「執行時間」本身不會有意義的差異（網路延遲、系統負載這些雜訊會蓋過真正的差異），所以這一課只量測「傳輸筆數」這個跟資料量成正比、不受雜訊影響的客觀指標，不聲稱做過具體的效能 Benchmark。傳輸筆數的差距是「資料量越大，把聚合工作留在資料庫做的優勢就越明顯」這個原則的具體證據，不是最終效能數字本身。

Eclipse ADT 建立 CDS View：Step by Step

1. 建 **ZI_CDS07_FLIGHT**：對著 **\$TMP** 套件右鍵 → New → Other ABAP Repository Object → **Data Definition** → Name **ZI_CDS07_FLIGHT** → Templates 選 **Define View (obsolete as of AS ABAP 7.57)** → Reference Object 選 **SFLIGHT** → 改成上面 Layer 1 內容 → Ctrl+S → Activate
2. 建 **ZC_CDS07_ROUTE_STATS**：同樣流程，不選 Reference Object（查的是 **ZI_CDS07_FLIGHT**，不是底層表），直接手動打完整內容 → Ctrl+S → Activate（注意 **Layer 1** 要先啟用成功）
3. 對 **ZC_CDS07_ROUTE_STATS** 用 **Data Preview**，確認每家航空公司只有一列，**FlightCount** / **TotalSeatsOccupied** / **AvgPrice** 都有正確的彙總數字

這一課學到的東西，接下來會怎麼用

- cds08（期中整合）：把聚合疊進最終的「航線營收分析」多層 View 案例
- 進階篇 cds11（Analytical Annotation 深入）：**@DefaultAggregation** 只是這一課的初探，cds11 會更完整介紹 **@Analytics.query**、Dimension vs. Measure 的正式標記方式

Eclipse ADT Step by Step (重點回顧)

1. **ZI_CDS07_FLIGHT** (明細層級，帶 **@DefaultAggregation** 提示)：以 **SFLIGHT** 為來源
2. **ZC_CDS07_ROUTE_STATS** (聚合層級)：以 **ZI_CDS07_FLIGHT** 為來源，**GROUP BY carrid**
3. Layer 1 先啟用，Layer 2 才能啟用
4. Data Preview 驗證每家航空公司只有一列彙總結果

學習目標

- 能寫出 **COUNT(*) / SUM(...) / AVG(...)** 聚合函數搭配 **GROUP BY** 的 CDS View
- 知道 **group by** 子句要寫在整個欄位清單 **{ }** 之後
- 能講出 **@DefaultAggregation** annotation 的用途 (給分析工具的聚合提示)，並知道它跟 **GROUP BY** (真正執行聚合) 是互補、不是同一件事
- 能講出 CDS 聚合相對「撈明細到應用層自己迴圈累加」的效能優勢，並知道這個優勢的根本原因 (傳輸筆數隨聚合層級大幅減少)
- 知道用小量測試資料驗證聚合正確性是合理的 (比對結果一致)，但不適合用來衡量真正的效能差異

物件清單

物件	名稱	型別
CDS Interface View (明細層級，帶聚合提示)	ZI_CDS07_FLIGHT	DDLS/DF
CDS Composite View (聚合層級)	ZC_CDS07_ROUTE_STATS	DDLS/DF
驗證程式	ZR_CDS07_DEMO	PROG/P

全部物件都在 **\$TMP** 套件，已建立並啟用 (讀取 active 版本內容與寫入內容逐字相符)。

動手練習物件 (你自己動手建，名稱自訂，不算在上面正式的課程物件清單裡)：

物件	建議名稱	型別	對應練習
CDS Composite View (依 carrid+connid 分組)	ZC_CDS07_CONNECTION_STATS (自訂)	DDLS/DF	動手練習

動手練習

輪到你了：疊一個新的聚合 View 在既有的 **ZI_CDS07_FLIGHT** (已經建好不用重建) 之上，物件名稱、套件 **\$TMP**，命名自訂 (例如 **ZC_CDS07_CONNECTION_STATS**)，要求：

1. **GROUP BY** 改成 **carrid 加上 connid** (也就是「依航線分組」，不只依航空公司分組——同一家航空公司可能有很多條不同航線)
2. 至少用三種聚合函數 (**COUNT / SUM / AVG**，門檻/欄位可以自己選)
3. 用 Data Preview 驗證：確認同一家航空公司如果有多條航線，會出現多列 (跟這一課範例「每家航空公司只有一列」不一樣)

建好、啟用成功後跟我說一聲，我會幫你核對語法。

如果你想額外挑戰一下：試著故意在欄位清單裡放一個沒有出現在 **GROUP BY**、也沒有被聚合函數包起來的欄位（例如直接放 **seatsmax** 而不包 **sum()/avg()**），實際看一次啟用失敗的錯誤訊息——親眼看過這個錯誤，比只是讀講義印象更深。

驗證方式

ZR_CDS07_DEMO 透過 **programrun** 無頭驗證，比較 CDS 聚合結果跟應用層手動迴圈累加的結果是否一致，並量測兩種做法各自傳輸的資料筆數：

```
=== 1. CDS Aggregation: ZC_CDS07_ROUTE_STATS (carrid = AA) ===
FlightCount:          25
TotalSeatsOccupied:    5,712
AvgPrice:  4.229400000000000E+02
=== 2. Application-layer aggregation: fetch raw rows, loop in ABAP ===
FlightCount:          25
TotalSeatsOccupied:    5,712
AvgPrice:             422.94
=== 3. Do CDS aggregation and application-layer loop agree? ===
MATCH: CDS aggregation and manual application-layer loop produce identical results
=== 4. Row counts moved across the network: CDS approach vs. application-layer
approach ===
CDS aggregation approach: rows transferred to application server =          8 (one
row per carrier)
Application-layer approach: rows transferred to application server =          25 (one
row per flight, for AA alone)
```

FlightCount (25)、**TotalSeatsOccupied** (5,712)、**AvgPrice** (約 422.94，兩種算法只是顯示格式不同，數值一致) 兩種算法完全吻合；筆數比較部分，**ZC_CDS07_ROUTE_STATS** (已經聚合過，橫跨所有航空公司) 只需要傳回 8 列，光是應用層要自己算 **AA** 一家航空公司的明細就要傳 25 列——證實聚合層級 **View** 大幅減少了需要搬到應用層的資料量。

動手練習的驗證方式：Eclipse 啟用成功 + 用 Data Preview 看查詢結果，貼程式碼給我核對語法。

思考題

- 如果把 **ZC_CDS07_ROUTE_STATS** 的 **GROUP BY** 只依 **carrid** 改成不分組（把整個 **GROUP BY** 子句拿掉，但保留聚合函數），你覺得會發生什麼事？（提示：想想看「沒有分組依據的聚合」在語意上代表什麼）
- @DefaultAggregation: #SUM** 標在 **ZI_CDS07_FLIGHT** 的 **seatsocc** 欄位上，如果之後有人把它改成 **@DefaultAggregation: #AVG**，你覺得會不會影響 **ZC_CDS07_ROUTE_STATS** 裡 **sum(seatsocc)** 這行的計算結果？（提示：回顧「兩者是互補、不是同一件事」那一節）
- 這一課驗證的是「小量資料下，CDS 聚合結果正確」跟「傳輸筆數確實比較少」，但沒有實際量測執行時間差異。如果要你設計一個真正能看出時間差異的實驗，你會怎麼做？（提示：想想看資料量要多大、要怎麼排除網路延遲等雜訊）
- 如果 **ZC_CDS07_ROUTE_STATS** 疊在一個有 **#CHECK** 存取限制的 Interface View 之上（像 **cds06** 的 **ZI_CDS06_FLIGHT**），你覺得聚合出來的 **FlightCount** 這類數字，會不會反映出「權限受限後」的資料，還是會反映出「完整、未受限」的資料？

答案

見 `zi_cds07_flight.ddls.abap`、`zc_cds07_route_stats.ddls.abap`、`zr_cds07_demo.prog.abap`。

SAP 端物件：`ZI_CDS07_FLIGHT`（明細層級）、`ZC_CDS07_ROUTE_STATS`（聚合層級）、`ZR_CDS07_DEMO`（驗證程式）。動手練習（依航線分組的聚合 View）由你在 Eclipse 動手建立，沒有固定答案快照——建好後跟我核對即可。